

Business Requirements Document (BRD)

Project: Orders and Customer Data Ingestion, SCD Type 2 History, and Aggregate Spend Reporting

Prepared by: Senior Business Analyst

Date: 2026-03-29

Version: 1.0

1. Executive Summary

- Purpose: Define business and functional requirements to ingest customer and order CSV source data into Delta tables, perform data cleansing and transformation, maintain an ordersummary as a Slowly Changing Dimension (SCD) Type 2 to capture customer change history, and compute daily customer aggregate spend. The solution will be implemented using Delta Live Tables (DLT) on the Databricks platform and stored in the gen_ai_poc_databrickscoe catalog, sdlc_wizard schema.
- Objectives:
- Reliable ingestion of source CSVs into Delta customer and order tables.
- Data quality: removal of nulls and duplicates.
- Enrich order data with TotalAmount (PricePerUnit * Qty).
- Maintain ordersummary as SCD Type 2 with full history when customer data changes.
- Provide daily per-customer aggregate spend (customeraggregatespend).
- Use Delta Live Tables for orchestration, reliability, streaming/batch support, and lineage.

2. Scope

- In scope:
- Read source CSV files from paths:
- /Volumes/gen_ai_poc_databrickscoe/sdlc_wizard/customerdata (customer)
- /Volumes/gen_ai_poc_databrickscoe/sdlc_wizard/orderdata (order)
- Create Delta tables: customer (staging), order (staging), ordersummary (SCD Type 2), customeraggregatespend.
- Transform: compute TotalAmount on orders, remove NULL values and duplicate records.
- Implement SCD Type 2 behavior on ordersummary keyed by CustId (and OrderId to link orders).
- Aggregation: sum TotalAmount by Name and Date into customeraggregatespend.
- Implementation and orchestration using Delta Live Tables.
- Out of scope:
- Changes to source systems that produce CSVs.
- Real-time stream ingestion technologies besides DLT (unless DLT streaming features are used).
- Downstream BI/reporting development, beyond creation of customeraggregatespend table.

- GDPR/PII governance beyond basic data handling controls (recommendations provided in Non-functional).

3. Stakeholders

- Business Sponsor: Head of Sales/Customer Insights
- Product Owner: Data Product Manager (Orders/Customer)
- Data Engineering: Delta/DLT engineers, ETL developers
- Data Modeling: Enterprise Data Architect
- Data Quality & Governance: Data Steward
- Analytics/BI: Reporting Analysts
- Platform/DevOps: Databricks Admins

4. Business Drivers and Benefits

- Business drivers:
 - Need to maintain historical view of customer attributes over time (e.g., Name, EmailId, Region) and preserve order records tied to the correct version of customer attributes.
 - Provide daily aggregated spend per customer for analytics and reporting.
 - Standardize ingestion and transformation using Databricks Delta and Delta Live Tables.
- Benefits:
 - Accurate historical analytics for customer behavior and cohort analysis.
 - Simplified, repeatable pipeline with built-in quality and lineage.
 - Faster time to insight for marketing/finance through ready-to-consume aggregate table.

5. Assumptions

- Source CSV files adhere roughly to the provided schema; minor schema changes may occur (column renames, types).
- Files are available in the specified volume path with predictable naming/partitioning.
- CustId is the stable identifier for customers. OrderId uniquely identifies orders.
- Databricks environment is provisioned and accessible with appropriate privileges to create catalogs/schemas/tables in gen_ai_poc_databrickscoe.sdmc_wizard.
- DLT is available and approved for production use.

6. Business and Functional Requirements

6.1 Ingestion Requirements

- FR-1: Read customer CSV data from /Volumes/gen_ai_poc_databrickscoe/sdmc_wizard/customerdata and load into a Delta table named customer (staging).

- FR-2: Read order CSV data from /Volumes/gen_ai_poc_databrickscoe/sdlc_wizard/orderdata and load into a Delta table named order (staging).
- FR-3: Ingestion should be idempotent—reprocessing the same files should not create duplicates in staging Delta tables.

6.2 Source Schemas (expected)

- Customer staging schema:
 - CustId (string / integer) — primary business key for customer
 - Name (string)
 - EmailId (string)
 - Region (string)
- Order staging schema:
 - OrderId (string / integer) — unique order identifier
 - ItemName (string)
 - PricePerUnit (decimal / double)
 - Qty (integer)
 - Date (date / timestamp)
 - CustId (string / integer) — foreign key referencing customer.CustId

6.3 Transformation Requirements

- FR-4: Enrich order staging by adding column TotalAmount = PricePerUnit * Qty. Ensure numeric precision is sufficient for financial amounts.
- FR-5: Remove rows with NULL/NULL-like values in critical fields before loading downstream. Critical fields:
 - For customer: CustId, Name (if Name is mandatory), EmailId (if required), Region (if required). The business must confirm which fields are mandatory; default: CustId mandatory, others nullable but flagged.
 - For order: OrderId, CustId, PricePerUnit, Qty, Date.
- FR-6: Remove duplicate records from customer and order staging tables. Duplicates defined as same values across all columns (staging). For orders, also enforce uniqueness by OrderId.
- FR-7: Validate and coerce data types (dates, numerics). Invalid type conversions should route records to a quarantine/error table with error metadata.

6.4 Ordersummary (SCD Type 2) Requirements

- FR-8: Create ordersummary table in catalog gen_ai_poc_databrickscoe, schema sdlc_wizard if it does not exist.
- FR-9: ordersummary schema must contain at minimum:
 - CustId
 - Name

- EmailId
- Region
- OrderId
- ItemName
- PricePerUnit
- Qty
- Date
- TotalAmount (computed)
- SCD metadata columns required for Type 2:
- IsActive (boolean) — indicates the current active record
- StartDate (timestamp/date) — when the version became active
- EndDate (timestamp/date, nullable) — when the version was superseded; NULL for active
- VersionNumber or HashKey (optional) — to track sequence or detect changes
- RowHash (optional) — to detect change in payload
- EffectiveTimestamp and LoadTimestamp (optional)
- FR-10: Load ordersummary by joining customer and order on CustId and include TotalAmount on each row.
- FR-11: Maintain historical versions per customer/order such that if customer attributes change (e.g., Name, EmailId, Region), prior rows associated with those orders remain preserved with EndDate set and IsActive = false, and a new row inserted with updated customer attributes and IsActive = true and EndDate = NULL.

6.5 SCD Type 2 Specific Logic

- FR-12: Detection of change:
- For each (OrderId) or logical order row, compare the current customer attribute values joined at load time with the latest active ordersummary attributes for that order (or the customer's existing orders). A change is defined as any difference in tracked attributes (Name, EmailId, Region).
- Implement row-level hashing or explicit column comparison to detect changes efficiently.
- FR-13: When a change is detected:
- Update the existing ordersummary record(s) for affected order(s) and customer to set IsActive = false and EndDate = .
- Insert new record(s) with latest customer attributes, IsActive = true, StartDate = , EndDate = NULL.
- FR-14: Ensure full history is maintained: do not delete old records; mark inactive instead.
- FR-15: Handle new orders and new customers:
- New orders: insert into ordersummary with IsActive = true and StartDate assigned.
- New customers (new CustId): insert new orders with appropriate attributes.
- FR-16: If an order record changes (e.g., Qty or PricePerUnit changed), treat that as an update for that order row and preserve history similarly (create new SCD row for the order with new order

attributes). Business must confirm which columns trigger SCD for order-level changes versus only customer-level changes.

- FR-17: Ensure referential integrity between ordersummary and customer aggregations (customeraggregatespend).

6.6 Aggregation Requirements

- FR-18: Create (if not exists) customeraggregatespend table in gen_ai_poc_databrickscoe.sdmc_wizard with columns:
 - Name (string)
 - TotalAmount (decimal)
 - Date (date)
 - Additional optional metadata: LoadDate, Currency, AggregationGranularity
- FR-19: Daily aggregation logic:
 - Source: ordersummary (active and/or historical? Business to confirm; default: sum across ordersummary records where Date = target Date — typically use the ordersummary record versions applicable to that Date or sum across orders by Date).
 - Group by Name and Date.
 - Aggregate TotalAmount using SUM(TotalAmount).
 - Load result into customeraggregatespend (idempotent upsert/overwrite behavior for same Name+Date).
- FR-20: Ensure aggregation handles active records appropriately. Business must confirm whether aggregates should use current Name for historical Date (if names change) or use the Name value valid at the order Date. Recommended: use the Name from the ordersummary row version whose StartDate/EndDate covers the order.Date — i.e., accurate historical attribution.

6.7 Implementation Platform Requirements

- FR-21: Implement the pipeline using Delta Live Tables (DLT).
- Use DLT constructs to define sources, transformations, expectations, and targets.
- Use DLT's capabilities for incremental processing, lineage, data quality expectations, and automatic table management.
- FR-22: Use Delta tables for persistent storage with appropriate partitioning and clustering:
 - Partition ordersummary and customeraggregatespend by Date (or another appropriate key) for query performance.
 - Consider Z-ordering on CustId or OrderId for efficient lookups.
- FR-23: Ensure workflows can be scheduled or triggered automatically and support monitoring, alerting, and error handling provided by DLT.

7. Non-functional Requirements

- NFR-1: Performance

- Ingestion and transformation jobs should complete within agreed SLAs (to be defined with stakeholders). For typical daily loads, target end-to-end latency < 2 hours from file availability to loaded aggregates.
- NFR-2: Scalability
- Support increasing data volume (e.g., 10x current size) through autoscaling Databricks clusters and appropriate partitioning.
- NFR-3: Reliability/Idempotency
- Jobs must be idempotent and support re-runs without data duplication.
- NFR-4: Data Quality & Monitoring
- Implement DLT expectations and/or checks for:
 - Non-null critical fields.
 - Valid data types for numerics and dates.
 - Reasonable numeric bounds (e.g., Qty >= 0, PricePerUnit >= 0).
 - Duplicate detection.
- Route invalid rows to quarantine/error storage with metadata about failure reason and original payload.
- NFR-5: Security & Governance
- Ensure access controls via Unity Catalog / Databricks access controls for gen_ai_poc_databrickscoe catalog and sdhc_wizard schema.
- Encrypt data at rest and in transit per platform defaults.
- Log data access and changes for audit.
- NFR-6: Recoverability
- Enable time travel and Delta table versioning for recovery and audit.
- NFR-7: Observability
- Implement job-level metrics, lineage, and alerts to notify stakeholders on failures, data quality issues, or SLA breaches.

8. Data Model and Schema Definitions

- Staging: customer (Delta)
- CustId: STRING (PK)
- Name: STRING
- EmailId: STRING
- Region: STRING
- IngestionMetadata: {FileName, IngestTimestamp, SourceHash} (optional)
- Staging: order (Delta)
- OrderId: STRING (PK)
- ItemName: STRING

- PricePerUnit: DECIMAL(18,2)
- Qty: INTEGER
- Date: DATE
- CustId: STRING (FK -> customer.CustId)
- TotalAmount: DECIMAL(18,2) — computed
- IngestionMetadata: {FileName, IngestTimestamp} (optional)
- Persistent: ordersummary (Delta, SCD Type 2)
- RowId: UUID or surrogate key
- CustId: STRING
- Name: STRING
- EmailId: STRING
- Region: STRING
- OrderId: STRING
- ItemName: STRING
- PricePerUnit: DECIMAL(18,2)
- Qty: INTEGER
- Date: DATE
- TotalAmount: DECIMAL(18,2)
- IsActive: BOOLEAN
- StartDate: TIMESTAMP
- EndDate: TIMESTAMP (nullable)
- VersionNumber: INTEGER (optional)
- RowHash: STRING (optional)
- LoadTimestamp: TIMESTAMP
- Persistent: customeraggregatespend (Delta)
- Name: STRING
- Date: DATE
- TotalAmount: DECIMAL(18,2)
- LoadDate: TIMESTAMP (optional)

9. Data Flow and Process Overview (High-level Steps)

- Step 1: Source ingestion
- Read customer CSV from /Volumes/.../customerdata into DLT-managed Delta table customer (staging).
- Read order CSV from /Volumes/.../orderdata into DLT-managed Delta table order (staging).

- Step 2: Schema enforcement & cleansing
- Validate and coerce data types.
- Remove rows with NULLs in critical fields (log quarantined rows).
- Deduplicate rows (by full row for staging; by OrderId for orders).
- Step 3: Enrichment
- Compute TotalAmount = PricePerUnit * Qty and add to the order staging table.
- Step 4: Create ordersummary table if not exists.
- Step 5: Merge/Upsert into ordersummary using SCD Type 2 logic
- Join customer and order on CustId to produce enriched order records.
- For each enriched order:
 - If there is no existing ordersummary row for that OrderId and attributes, insert new active record.
 - If existing active record has different customer/order attribute values (RowHash differs), set existing record IsActive = false and EndDate = now(), then insert new record with updated attributes and IsActive = true (StartDate = now()).
- Ensure merges are atomic / ACID using Delta Merge.
- Step 6: Build customeraggregatespend
- Aggregate ordersummary records (preferably using the order.Date and the ordersummary row version valid for that order.Date) grouped by Name and Date.
- Upsert the aggregated values into customeraggregatespend, replacing prior entries for the same Name+Date.
- Step 7: DLT orchestration
- Implement all steps as DLT pipeline(s) with clear lineage, expectations, and quality checks.
- Configure schedules/triggers and alerting.

10. Business Rules and Edge Cases

- BR-1: Identifiers
- CustId is primary customer key. If CustId is missing, route to quarantine.
- OrderId is primary order key. If OrderId duplicates with conflicting payloads, treat as update and apply SCD/merge logic; log conflict if ambiguous.
- BR-2: Null handling
- Critical nulls cause quarantine. Non-critical nulls allowed but flagged.
- BR-3: Duplicate handling
- Exact duplicates removed at staging; near duplicates (same OrderId but different attributes) are handled as updates.
- BR-4: Date handling
- Date parsing should be robust and support common formats; invalid dates route to quarantine.
- BR-5: Currency/precision

- PricePerUnit and TotalAmount should maintain 2 decimal places. Business to confirm currency; if multiple currencies present, include Currency field and convert or segregate by currency.
- BR-6: Late-arriving data
- Late-arriving orders (order Date earlier than processing window) must be processed; aggregations should be recomputed for affected Date partitions.
- BR-7: Soft deletes
- If a customer record is removed from source, SCD Type 2 handling must be defined: mark all current active ordersummary records for that customer as inactive and insert or update as per business definition. Default: no automatic deletion; data steward must define delete semantics.
- BR-8: Multiple orders per customer and SCD granularity
- SCD Type 2 granularity is order-level rows combining customer attributes. When a customer attribute changes, all orders associated with that customer may need new SCD rows or only future orders — business must confirm expected semantics. Recommended: preserve historical correctness by associating each order with customer attributes as of the order's processing time or the version applicable at the order date.

11. Data Quality, Testing and Validation

- Test cases to validate:
- Ingest sample CSVs and verify staging tables populated.
- Verify TotalAmount calculation correctness across sample rows (including decimals).
- Verify null removal and duplicates elimination behavior.
- SCD Type 2 scenarios:
- Update to customer Name — ensure prior ordersummary records are marked inactive and new ones inserted.
- Update to order attributes — ensure history preserved.
- Insert new customer and new order — ensure creation of active rows.
- Aggregation:
- Validate daily sums by Name and Date reflect ordersummary source.
- Negative tests:
- Invalid data types, missing critical fields — verify quarantine mechanics.
- Idempotency:
- Re-run ingestion for same files and verify no duplicates.
- Data quality expectations/metrics:
- % of quarantined records per run (thresholds to be agreed).
- Freshness SLA (e.g., 95% of data processed within SLA).
- Number of merge conflicts per run.

12. Security, Privacy and Compliance

- Data access limited to authorized roles via Unity Catalog and Databricks permissions.

- PII handling (EmailId): define retention, masking, or encryption policies with data governance.
- Maintain audit logs of changes to ordersummary and aggregate tables.
- If data contains regulated PII, apply additional controls (masking, restricted exports).

13. Operational Considerations and Runbook

- Monitoring:
 - Use DLT monitoring UI and alerts for failures and quality expectation violations.
 - Implement job-level Slack/email alerts to Data Engineering and Data Steward on critical failures.
- Recovery:
 - For failure during merge/upsert, use Delta time travel and table version history to revert or re-run.
- Maintenance:
 - Periodic vacuuming per Delta retention policy.
 - Catalog and table permissions review quarterly.
- Incremental deployments:
 - Use feature flags or phased rollout when changing SCD logic.
 - Support contacts and escalation matrix to be defined outside this BRD.

14. Implementation Plan & Timeline (High-level)

- Phase 0 — Kickoff & environment setup (1 week)
 - Validate access to data paths and Databricks workspace.
 - Confirm business rules and required mandatory fields.
- Phase 1 — Staging & ingestion development (1–2 weeks)
 - Implement DLT pipeline to read CSVs into staging, enforce schema, dedupe, and compute TotalAmount.
- Phase 2 — SCD Type 2 design & implementation (2 weeks)
 - Implement ordersummary schema, merge SCD Type 2 logic, and test scenarios.
- Phase 3 — Aggregation & customeraggregatespend (1 week)
 - Implement daily aggregation, idempotent upserts.
- Phase 4 — QA, performance tuning, and data validation (1–2 weeks)
 - End-to-end testing, QA signoff, DLT expectation tuning.
- Phase 5 — Production rollout and monitoring (1 week)
 - Deploy DLT pipelines to production, configure monitoring and runbooks.
- Total estimated: 6–9 weeks (subject to resource availability and change requests).

15. Risks and Mitigations

- Risk: Source CSV schema changes break ingestion.

- Mitigation: Implement schema evolution handling in DLT, maintain clear versioning and alerts for schema drift.
- Risk: Misinterpretation of SCD scope (customer vs order changes).
- Mitigation: Conduct business workshop to finalize which columns trigger SCD, and document examples.
- Risk: Late-arriving data impacting aggregates.
- Mitigation: Implement reprocessing logic to recompute affected Date partitions and incremental backfill processes.
- Risk: Performance at scale.
- Mitigation: Partitioning, Z-ordering, and cluster sizing; perform load testing.

16. Open Questions / Decisions Needed

- Q1: Which customer fields are mandatory? Confirm which fields trigger quarantine if NULL.
- Q2: Does SCD Type 2 apply per-order or per-customer? Specifically, when a customer attribute changes, should all historical orders be re-associated with new values or should only future orders use new values? Recommended: SCD should preserve order-level historical correctness (use version applicable at order time).
- Q3: Should customeraggregatespend aggregate based on the Name value at the time of the order or current Name? Recommended: use Name from ordersummary version valid for the order.Date.
- Q4: Currency: is data single-currency or multi-currency? If multi-currency, define currency handling (conversion, separate aggregates).
- Q5: Error handling SLA and retention policy for quarantine/error records.
- Q6: Retention policy for historical SCD rows (keep forever or TTL?). Consider legal/compliance constraints.

17. Acceptance Criteria

- AC-1: Customer and order CSV ingestion sets up staging Delta tables with correct schema and metadata.
- AC-2: TotalAmount column is present and correct for all order rows.
- AC-3: No duplicates or critical NULLs exist in staging after DLT expectations are applied (apart from quarantined rows).
- AC-4: ordersummary exists and implements SCD Type 2 semantics: when customer attribute changes, previous record(s) are marked inactive and new record(s) inserted with StartDate and EndDate appropriately set.
- AC-5: customeraggregatespend contains correct SUM(TotalAmount) grouped by Name and Date and is idempotent across re-runs.
- AC-6: Pipeline implemented using Delta Live Tables with expectations, monitoring, and documented runbook.
- AC-7: QA test cases pass per Data QA signoff and stakeholders approve.

18. Deliverables

- DLT pipeline code/scripts implementing steps 1–10.
- Delta tables: customer (staging), order (staging), ordersummary (SCD Type 2), customeraggregatespend.
- Data quality expectations and quarantine/error handling mechanisms.
- Documentation:
 - Technical design document including merge logic/queries.
 - Runbook and monitoring configuration.
 - Test plan and test results.
 - Access and security configuration notes.

19. Appendix — Technical Implementation Notes (suggested)

- Use Delta MERGE INTO for SCD Type 2 changes:
- Compute a hash of tracked attributes (RowHash) for both staging and target to detect changes quickly.
- Merge logic pseudocode:
 - WHEN matched AND RowHash differs AND IsActive = true -> UPDATE set IsActive=false, EndDate=now()
 - WHEN not matched -> INSERT new row with IsActive=true, StartDate=now(), EndDate=NULL
- For order-level changes, use OrderId as unique matching key; for customer-level changes that affect multiple orders, identify all orders for the CustId and apply SCD update per business rule.
- Use DLT expectations (e.g., dlt.expect) to validate non-null and range conditions.
- Partition recommendations:
 - ordersummary: partition by Date (e.g., order_date) or by year/month to balance partition sizes.
 - customeraggregatespend: partition by Date.
- Maintain audit columns:
 - load_timestamp, source_file_name, batch_id.
- Testing:
 - Unit tests using synthetic datasets.
 - Integration tests with DLT pipeline runs in controlled environment.
- Performance:
 - Use optimized write modes (e.g., write to temporary table and MERGE).
 - Use adaptive execution and autoscaling clusters.

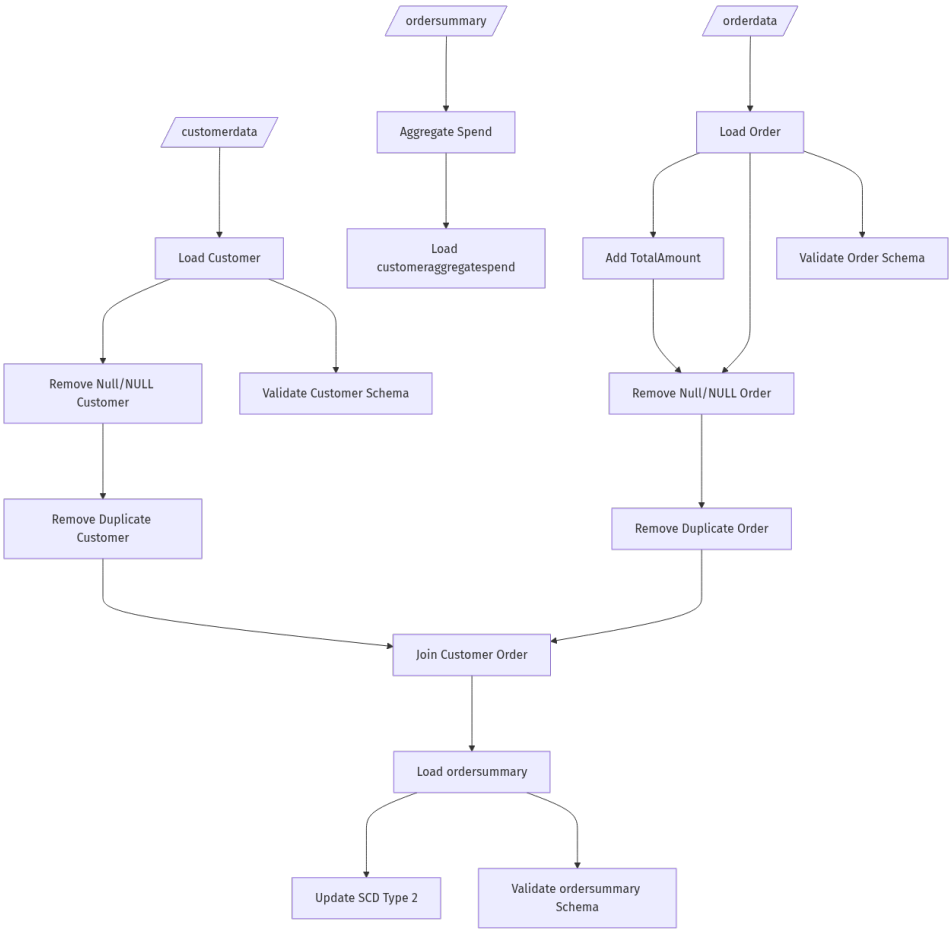
If you would like, next steps I can take:

- Produce a detailed technical design document with Delta MERGE SQL or PySpark/DLT code samples for each step.
- Create a DLT pipeline specification with exact expectations and code templates.

- Draft test cases and sample test data for QA runs.
- Facilitate a workshop with stakeholders to answer open questions and finalize SCD granularity and mandatory fields.

Please indicate which deliverable you want next (e.g., technical design, DLT pipeline code, test plan, or a stakeholder workshop agenda).

Data Flow Diagram (DFD)



```mermaid

graph TD;

CustomerData[/customerdata/] ---> LoadCustomer[Load Customer];

OrderData[/orderdata/] ---> LoadOrder[Load Order];

LoadCustomer ---> RemoveNullCustomer[Remove Null/NULL Customer];

LoadOrder ---> AddTotalAmount[Add TotalAmount];

LoadOrder ---> RemoveNullOrder[Remove Null/NULL Order];

AddTotalAmount ---> RemoveNullOrder;

RemoveNullCustomer ---> RemoveDuplicateCustomer[Remove Duplicate Customer];

RemoveNullOrder ---> RemoveDuplicateOrder[Remove Duplicate Order];

RemoveDuplicateCustomer ---> JoinCustomerOrder[Join Customer Order];

RemoveDuplicateOrder ---> JoinCustomerOrder;

JoinCustomerOrder ---> LoadOrderSummary[Load ordersummary];

LoadOrderSummary ---> UpdateSCDType2[Update SCD Type 2];

ordersummary[/ordersummary/] ---> AggregateSpend[Aggregate Spend];

AggregateSpend ---> LoadCustomerAggregateSpend[Load customeraggregatespend];

LoadCustomer ---> ValidateCustomer[Validate Customer Schema];

LoadOrder ---> ValidateOrder[Validate Order Schema];

LoadOrderSummary ---> ValidateOrderSummary[Validate ordersummary Schema];

...